

Universidad Nacional Autónoma de México

Facultad de Ciencias

Cómputo Concurrente

2027-1

Tarea 4. Fundamentos de memoria compartida: Registros y Snapshots

ALUMNOs:

Martínez Hidalgo Paola Mildred - 319300217
Dominguez Sanchez Gustavo Alonso - 418047121

Fecha de entrega:
21 de octubre de 2026

Profesor:
Gilde Valeria Rodríguez



Ejercicios

1. **[15 min/12.5 puntos]** De acuerdo a la clase y al siguiente video `https://youtu.be/iAqyufwYcHc?si=rRdjISdUndNSakb9` (también puedes consultar el libro principal en el capítulo 4), contesta de forma breve:

   a) ¿Cuál es la diferencia entre un registro regular y uno seguro?

   En un registro seguro (Safe Registrer) si la lectura no se solapa con ninguna escritura este devolverá el último valor escrito, pero si la lectura se solapa con una o más escrituras puede que el registro devuelva cualquier valor dentro del rango de valores permitidos por el registro. En otras palabras puede que nos devuelva valores basura, es decir, valores que nunca se escribieron.

   Por otra parte el registro regular (Regular Registrer) al igual que el seguro si la lectura no se solapa con ninguna escritura nos va a devolver el último valor escrito. La diferencia justo está en que si la lectura se solapa con una o más escrituras no va a devolver valores arbitrarios (valores basura), si no que únicamente puede devolver el valor que tenía el registro antes de la escritura o el valor de alguna de las escrituras que se están ejecutando concurrentemente con la lectura.

   b) ¿Cuál es la diferencia entre un registro atómico y uno regular?

   Ambos registros evitan devolver valores basura, la diferencia está en que el registro regular permite la **inversión nuevo/viejo (new/old inversion)**, con esto nos referimos a que si dos lecturas consecutivas (donde la segunda ocurre estrictamente después de la primera) se solapan con una escritura, es posible que la primera lectura devuelva el nuevo valor que se está escribiendo y que la segunda lectura devuelva el valor antiguo.

   En cambio el registro atómico (Atomic Registrer) es linealizable, por lo que prohíbe estrictamente las inversiones nuevo/viejo, en este caso se cumple con la condición de orden temporal

   $$R_1 \rightarrow R_2 \Longrightarrow stamp(R_1) \leq stamp(R_2)$$

   Con esto anterior nos referimos a que si una lectura inicial observa el nuevo valor escrito, ninguna lectura posterior podrá devolver el valor antiguo.

   c) ¿Qué relación hay entre los registros atómicos y el algoritmo de Bakery (desde un punto de vista histórico)?

   Antes de Lamport, se pensaba que para sincronizar hilos (exclusión mutua) el hardware debía ofrecer registros atómicos por defecto. Lamport diseñó el **Algoritmo de Bakery** (el cual se basó en la máquina que da turnos con papelitos en una panadería) y demostró que funcionaba correctamente usando únicamente **registros seguros**.

   Al intentar probar matemáticamente por qué funcionaba su algoritmo sobre memoria "no atómica", Lamport tuvo que inventar y formalizar la clasificación de registros **seguros**, **regulares** y **atómicos**.

   En el algoritmo de Bakery, para entrar a la sección crítica, cada hilo revisa los números de los demás, toma un número mayor (‘max + 1‘) y espera su turno. Años más

tarde, los científicos descubrieron que para construir un registro atómico con múltiples escritores (MRMW) a partir de registros sencillos, debían usar exactamente esa misma idea de etiquetas de tiempo (timestamps o tickets del Bakery) para ordenar qué escritura ocurrió antes y cuál después.

2. [30 min/25 puntos] Analiza la siguiente ejecución (figura 1) con respecto a la implementación del Snapshot Wait-free (listing 1). La inicialización del Snapshot es:
$SS = [\bot, \bot, \bot, \bot]$, en donde $SS[0]$ = registro de A, $SS[1]$ = registro de B, $SS[2]$ = registro de C y $SS[3]$ = registro de D.

Dados los siguientes valores de las vistas V_d y V_a, ¿es posible que la ejecución sea una ejecución del Snapshot Wait-free? Si tu respuesta es sí, argumenta porqué es linealizable con respecto a un objeto de tipo Snapshot atómico (muestra una linearización).

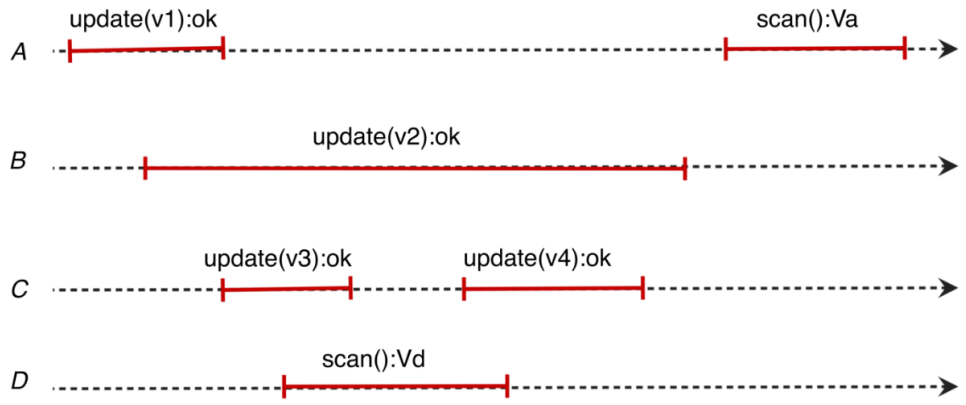


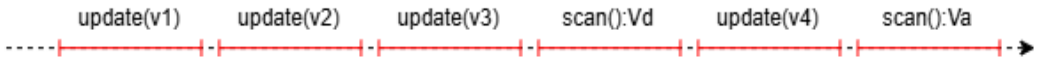
Figura 1: Ejecución

a) Si $V_d = [\bot, v_2, v_4, \bot]$ y $V_a = [v_1, v_2, v_4, \bot]$.
No, en este caso no es posible, esto lo podemos saber por la casilla 0 de V_d, *update(v1)* de A terminó antes de que D iniciara su *scan():Vd*, así que en la casilla 0 de V_d tendría que estar *v1* y no $\bot$.

b) Si $V_d = [v_1, v_2, v_4, \bot]$ y $V_a = [v_1, v_2, v_3, \bot]$.
No, en este caso no es posible. Al empezar por V_d se cumple correctamente, pero al pasar con V_a en la casilla 2 se genera un error, ya que tiene *v3*. Cuando A hace su scan ya terminaron todos los updates de C, C escribió primero v3 y después v4, así que la casilla 2 tendría que valer v4 y no v3, por lo tanto lo podemos descartar y decir que no cumple con una ejecución del Snapshot Wait-free.

c) Si $V_d = [v_1, v_2, v_3, \bot]$ y $V_a = [v_1, v_2, v_4, \bot]$.
En este inciso sí se trata de una ejecución del Snapshot Wait-free, para esto primero revisamos V_d, podemos ver que en la casilla 2 tenemos *v3*, en este caso no hay problema porque *v4* se traslapa con el scan de D, así que puede agarrar ya sea *v3* o *v4*. Después pasamos con V_a, que de igual forma no se encuentra ningún error a comparación de los que comentamos anteriormente.

Dado lo anterior pasamos a mostrar la linearización:



1) update(v1): Se linealiza durante su intervalo de ejecución. Estado del Snapshot: $[v1, \perp, \perp, \perp]$.
2) update(v2): Se linealiza dentro de su intervalo, justo antes de 'scan():Vd'. Estado del Snapshot: $[v1, v2, \perp, \perp]$.
3) update(v3): Se linealiza al inicio de su ejecución (concurrente con B y D). Estado del Snapshot: $[v1, v2, v3, \perp]$.
4) scan():Vd: Ocurre en su intervalo de tiempo. Devuelve el estado actual de la memoria $[v1, v2, v3, \perp]$, coincidiendo exactamente con el valor Vd del enunciado.
5) update(v4): Se linealiza dentro de su intervalo (después de scan():Vd y antes de scan():Va). Sobreescribe el registro de C. Estado del Snapshot: $[v1, v2, v4, \perp]$.
6) scan():Va: Se linealiza dentro de su intervalo. Devuelve el estado actualizado $[v1, v2, v4, \perp]$, coincidiendo con el valor Va.

Con lo anterior podemos concluir que la ejecución es completamente linealizable.

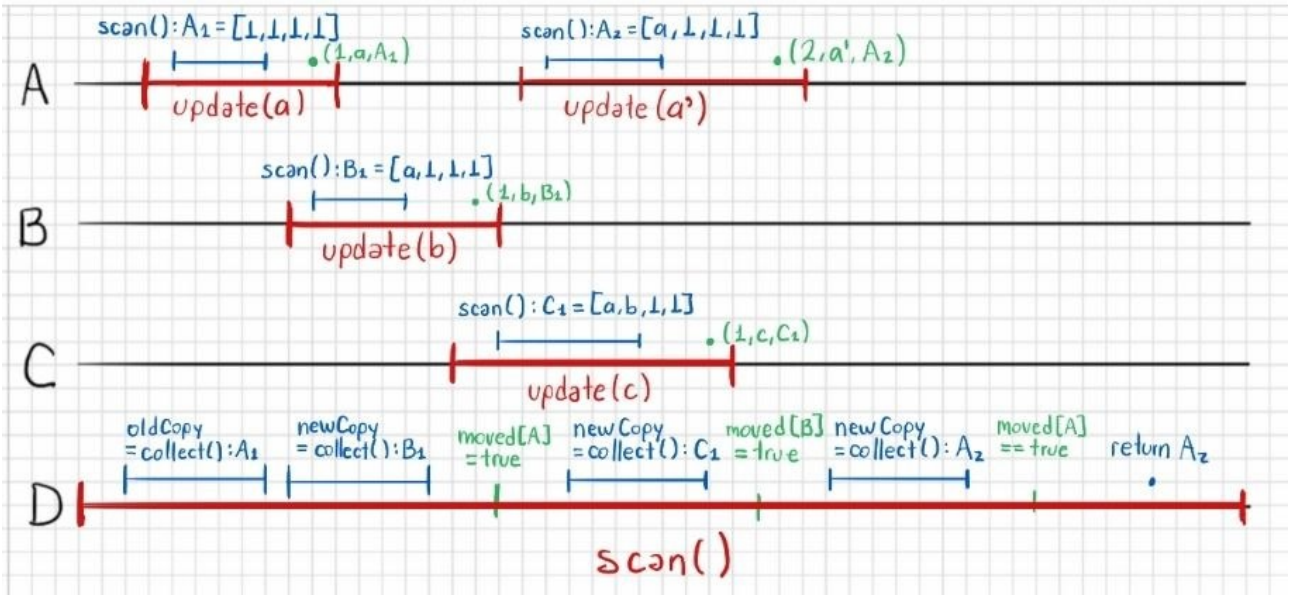
3. [20 min/25 puntos] Plantea una ejecución del Snapshot Wait-free (listing 1) para $n = 4$ hilos en donde muestres un ejemplo de porqué esta implementación es *bounded wait-free*. Es decir, que cada `update()` o `scan()` termina en a lo más $O(n^2)$ lecturas o escrituras (pasos).
Hint: Muestra uno de los peores escenarios en donde 4 llamadas `update()` *se traslapan.*

Listing 1: Snapshot Wait-free

```
1  public void update(T value) {
2      int me = ThreadID.get();
3      T[] snap = scan();
4      StampedSnap<T> oldValue = a_table[me];
5      StampedSnap<T> newValue =
6          new StampedSnap<T>(oldValue.stamp+1, value, snap);
7      a_table[me] = newValue;
8  }
9
10 public T[] scan() {
11     StampedSnap<T>[] oldCopy;
12     StampedSnap<T>[] newCopy;
13     boolean[] moved = new boolean[a_table.length];
14     oldCopy = collect();
15     collect : while(true) {
16         newCopy = collect();
17         for (int j = 0 ; j < a_table.length ; j++) {
18             if (oldCopy[j].stamp != newCopy[j].stamp) {
19                 if (moved[j]) {
20                     return oldCopy[j].snap;
21                 } else {
22                     moved[j] = true;
23                     oldCopy = newCopy;
24                     continue collect;
```

```
25                  }
26              }
27          }
28          T[] result = (T[]) new Object[a_table.length];
29          for (int j = 0 ; j < a_table.length ; j++)
30              result[j] = newCopy[j].value;
31          return result;
32      }
33  }
```

Para demostrar el límite del peor caso en la implementación de Wait-free, se diseña la siguiente traza donde un hilo se ve forzado a reiniciar su ciclo `while` el mayor número de veces posible antes de poder retornar.
Considérese un sistema con $n = 4$ hilos. Además que el arreglo de registros se inicializa en $[\bot, \bot, \bot, \bot]$. También recordemos que cada llamada a `collect()` requiere de $O(n)$ operaciones de lectura, ya que copia el arreglo completo de tamaño 4.



Entonces, en esta ejecución se muestran 4 llamadas `update()` que ocurren concurrentemente con la única llamada `scan()` del hilo D. Cada vez que el hilo D intenta verificar su lectura con un nuevo `collect()`, uno de los otros hilos modifica el arreglo, imposibilitando dar una Snapshot. Este es el peor escenario para $n = 4$ hilos por lo siguiente:

- El hilo D tiene un arreglo *moved* de tamaño 4.
- Para ser interrumpido el máximo de veces, se necesita que todos los demás hilos (A, B, C) interrumpan a A al menos una vez (haciendo las 3 llamadas `update()`.
- Para que el algorimto finalmente acabe y de una vista, necesita detectar que un hilo que ya se habia movido, se mueva por segunda vez. En este ejemplo, A ejecuta la cuarta llamada a `update()`.
- Al requerir así, 4 actualizaciones para terminar el ciclo `while`, el hilo D realiza un total de 4 iteraciones, y en cada iteración hace un `collect()`, que como mencionamos, le toma 4 operaciones, por lo que el total es de $4 \times 4 = 16 = 4^2$.

Esto muestra así que `update()` o `scan()` está acotado por $O(n^2)$.

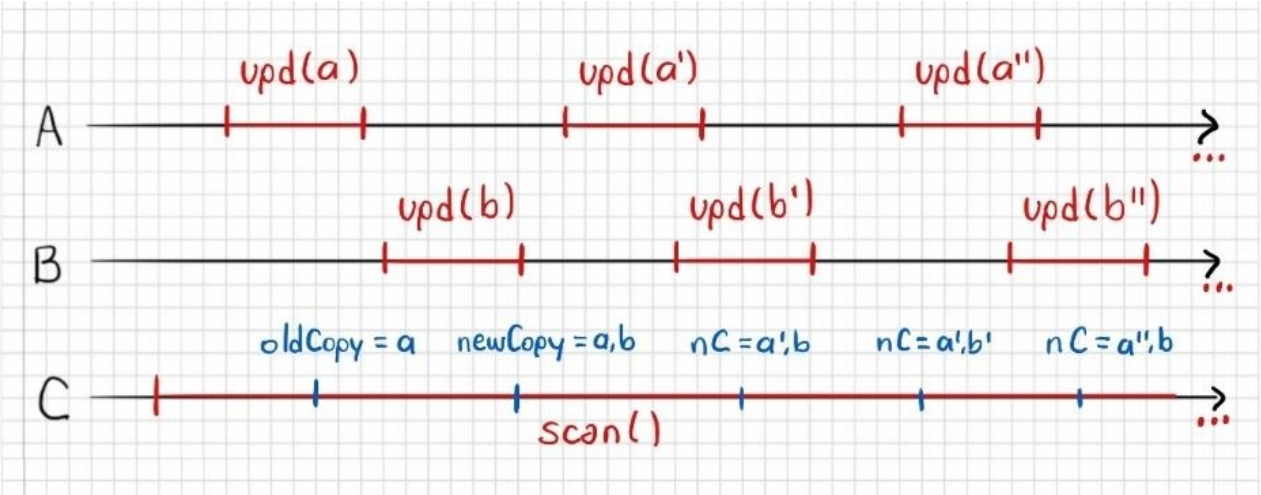
4. **[20 min/25 puntos]** Describe una ejecución del Snapshot Obstruction-free (listing 2) en la cual el método `scan()` de un hilo A nunca termina (considera $n = 3$ hilos).

Listing 2: Snapshot Obstruction-free

```
1  public class SimpleSnapshot<T> implements Snapshot<T>{
2      private StampedValue<T>[] a_table; // array of atomic MRSW registers
3      public SimpleSnapshot(int capacity, T init) {
4          a_table = (StampedValue<T>[]) new StampedValue[capacity];
5          for (int i = 0 ; i < capacity ; i++) {
6              a_Table[i] = new StampedValue<T>(init);
7          }
8      }
9      public void update(T value) {
10         int me = ThreadID.get();
11         StampedValue<T> oldValue = a_table[me];
12         StampedValue<T> newValue =
13             new StampedValue<T>((oldValue.stamp)+1, value);
14         a_table[me] = newValue;
15     }
16     private StampedValue<T>[] collect() {
17         StampedValue<T>[] copy = (StampedValue<T>[])
18             new StampedValue[a_table.length];
19         for (int j = 0 ; j < a_table.length ; j++)
20             copy[j] = a_table[j];
21         return copy;
22     }
23     public T[] scan() {
24         StampedValue<T>[] oldCopy, newCopy;
25         oldCopy = collect();
26         collect : while(true) {
27             newCopy = collect();
28             if(! Arrays.equals(oldCopy, newCopy)){
29                 oldCopy = newCopy;
30                 continue collect;
31             }
32             T[] result = (T[]) new Object[a_table.length];
33             for (int j = 0 ; j < a_table.length ; j++)
34                 result[j] = newCopy[j].value;
35             return result;
36         }
37     }
38 }
```

Para demostrar que esta implementación es obstruction-free en la cual un hilo nunca termina su `scan()`, mostramos la siguiente ejecución.

Consideremos $n = 3$ hilos, donde el hilo C ejecutará el `scan()` mientras que los hilos A y B ejecutan llamadas sucesivas a `update()`. Asumamos que el arreglo inicial de estampas es $[0, 0, 0]$.



- Notemos que el hilo C invoca `scan()`, y ejecuta su primer `oldCopy = collect()` antes del ciclo. Este `oldCopy` registra las estampas $[0, 0, 0]$.
- Justo antes de que el hilo C vuelva a leer, el hilo A invocó y completó un `update()`, ahora el arreglo de estampas global es $[1, 0, 0]$.
- El hilo C entra al ciclo `while` y ejecuta su `newCopy = collect()`. El hilo C evalúa que `oldCopy` y `newCopy` no son iguales. Por lo tanto, entra al condicional y se actualiza la vieja copia por la nueva. Luego, se ejecuta `continue collect` y reinicia el ciclo.
- Antes de que el hilo C vuelva a leer, el hilo B también invoca y completa un `update()`, cambiando el arreglo de estampas global.
- Nuevamente, el hilo C ve que al actualizar la nueva copia con `collect()`, este no es igual a la vieja copia. Entonces repite el proceso de actualizar la vieja copia y reiniciar el ciclo.
- Y así sucesivamente, el hilo A y B continuamente hacen `update()` justo antes de que C pueda realizar su lectura, por lo que `oldCopy` y `newCopy` nunca serán iguales y siempre se reiniciará el ciclo.

De esta forma el hilo C quedará atrapado en el ciclo `while` porque sus copias nunca serán idénticas. Por lo que el método `scan()` nunca devuelve un resultado y no termina.

5. **[15 min/12.5 puntos]** Si en la implementación del Snapshot Obstruction-free modificamos el método `scan()` de la siguiente manera:

Listing 3: `scan()` de Obstruction-free con solo un `collect()`

```
public T[] scan() {
```

```
1     copy = collect();
2     T[] result = T[] new Object(a_table.length);
3     for (int j = 0 ; j < a_table.length ; j++) {
4         result[j] = copy[j].value;
5     }
6     return result;
```

- ¿Podría suceder la siguiente implementación de la Figura 2? Supón que inicialmente el arreglo está inicializado: $[\bot, \bot, \bot]$.

  Si, es posible que suceda. Con la modificación que se hizo ahora el scan lee las casillas una por una sin ningún orden, tomando en cuenta esto podemos tener el siguiente caso:

  - Casilla 0 = $\bot$. El scan que se encuentra en B leyó temprano la casilla 0, antes de que A escribiera **c** y por lo tanto tendríamos la casilla vacía ($\bot$).
  - Casilla 1 = $\bot$. Como B no hace updates queda como $\bot$.
  - Casilla 2 = b. El scan leyó la casilla 2 después de que C escribió **b**.

  Con el caso anterior se cumple la implementación $V = [\bot, \bot, b]$.

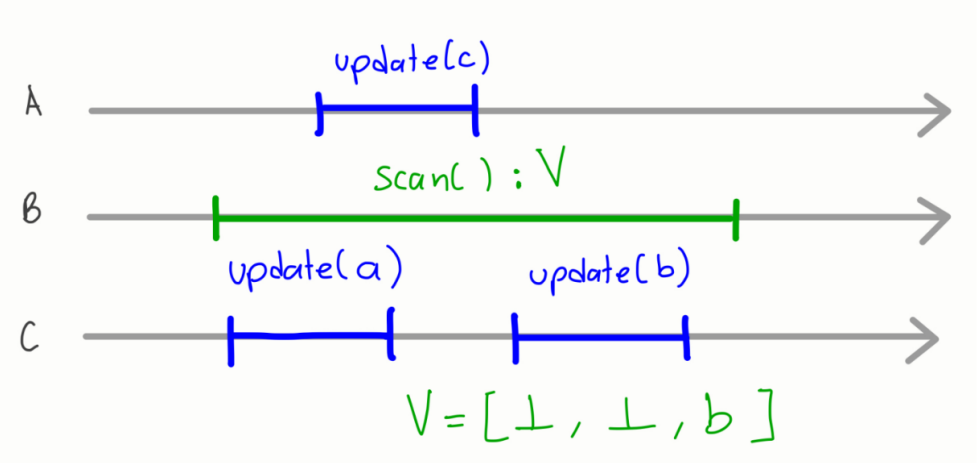


Figura 2:

- ¿La implementación seguirá siendo linealizable? Argumenta porqué.

  No, en este caso la implementación deja de ser linealizable. Para que la casilla 2 valga **b**, **update(b)** ya tuvo que haber ocurrido, pero notemos que **update(c)** termina antes de que **update(b)** empiece, por lo tanto en cualquier fila válida **update(c)** va antes de **update(b)**. Con esto anterior cuando la casilla 2 vale **b**, la casilla 0 tiene que valer **c**.

  Por lo tanto no podemos tener $V = [\bot, \bot, b]$, en el instante en que se tiene **b** tendríamos $V = [c, \bot, b]$. Con esto concluimos de que no es linealizable.